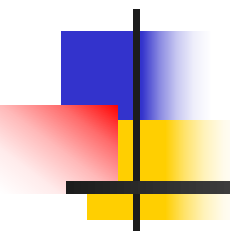


C语言程序设计

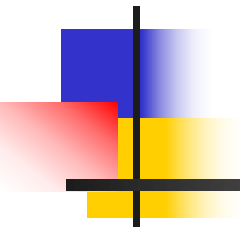
The C Programming Language



第10章 文件

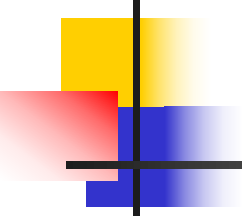
华中科技大学计算机学院

卢萍



文件

- ◆ 存储在变量和数组中的数据是临时的，程序结束后都会消失
- ◆ 文件用来永久地保存大量数据
- ◆ 文件被存储在外存储设备中（如硬盘）
- ◆ 本章讨论怎样用C程序建立、更新、处理数据文件



主要内容

■ 文件的打开与关闭

- fopen、fclose、freopen函数

■ 文本文件的读写

- fgetc、fputc、fgets、fputs、fprintf、fscanf 等函数

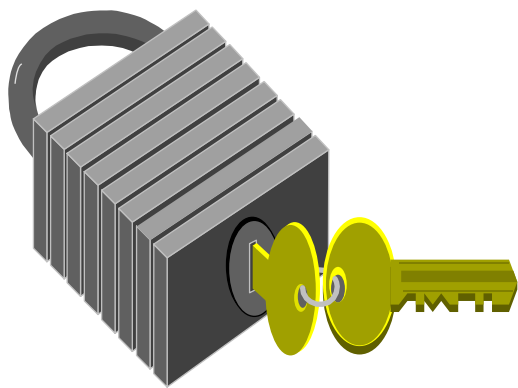
■ 二进制文件的读写

- fread、fwrite函数

■ 文件的随机读写

- fseek、rewind、ftell、fsetpos、fgetpos等文件指针定位函数

问：从键盘输入若干行字符，
保存到d:\a.txt文件中，该如何做？

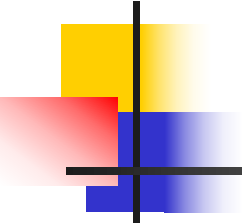


将输入写入
文件中

运行： `savefile.exe`

程序1

```
#include<stdio.h>
int main( )
{
    FILE *fp;
    char ch;
    if ( (fp = fopen("d:\\a.txt","w") ) == NULL){
        printf("can't open the file!");
        return -1;
    }
    while((ch = getchar( ) ) != EOF)
        fputc(ch,fp);
    fclose(fp);
    return 0;
}
```



文件操作的步骤

- (1) 打开文件——建立文件指针与文件间联系；
- (2) 通过文件指针对文件进行读写操作；
- (3) 关闭文件——取消文件指针与文件间的联系。

提示：打开文件时，确定了文件读写格式和读写方式！

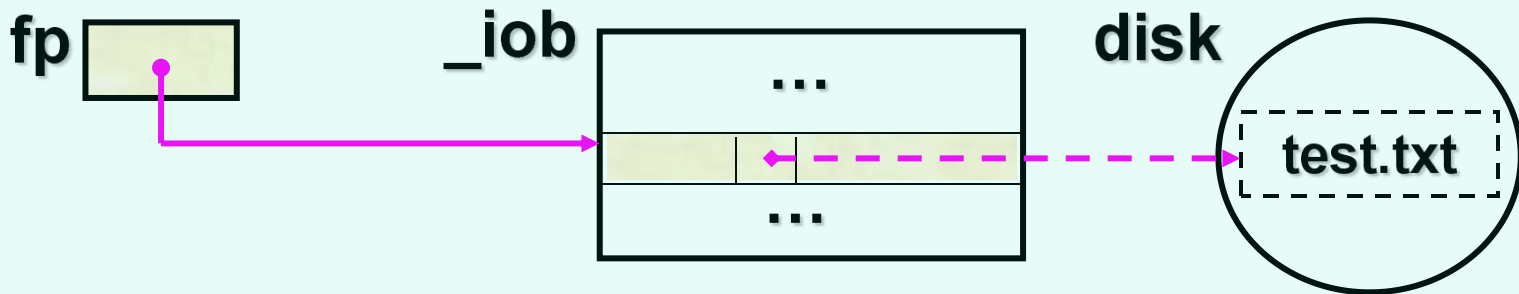
打开文件函数fopen()

```
FILE * fopen(const char * filename, const char * mode);
```

按照mode方式, 打开文件filename。最多可打开
FOPEN_MAX个文件
成功: 返回一个指向filename的文件指针
不成功: 返回空指针NULL

```
FILE *fp;
```

```
fp=fopen("c:\\test.txt", "w");
```



FILE结构

FILE是**stdio.h** 中定义的表示一个文件相关信息的结构类型。

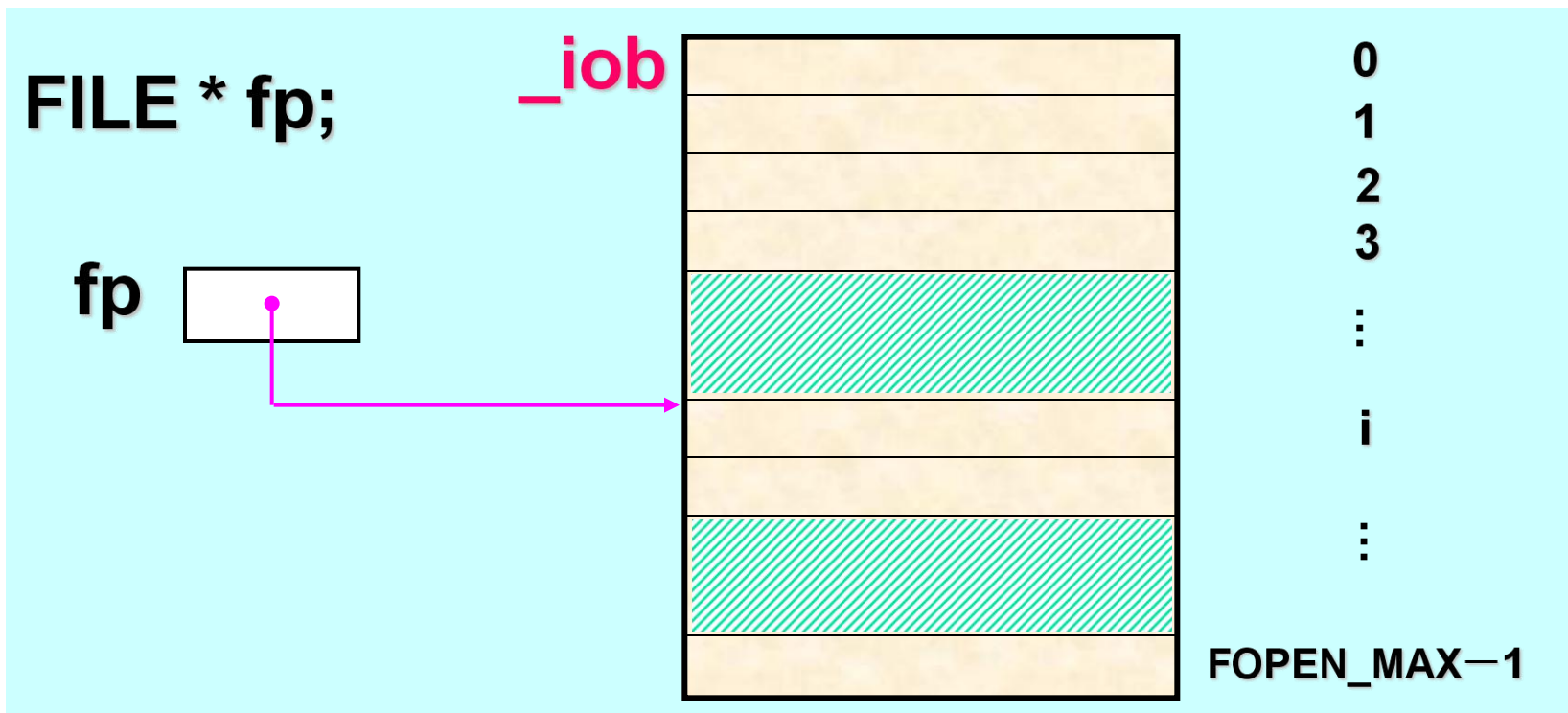
```
struct _iobuf {  
    char *_ptr;           //文件当前读写位置指针  
    int _cnt;             //当前缓冲区的相对位置  
    char *_base;          //文件的起始位置  
    int _flag;            //文件状态标志  
    int _file;            //文件描述符  
    int _charbuf;         //检查缓冲区状况,如果无缓冲区则不读取  
    int _bufsiz;          //文件缓冲区大小  
    char *_tmpfname;      //临时文件名  
};
```

typedef struct _iobuf FILE;

文件描述符是个简单的整数，用以标明每一个被打开的文件。第一个打开的文件是**0**，第二个是**1**，依此类推。

文件指针

程序中仅使用 `FILE *` 型变量，通常简称为 `FILE` 指针。



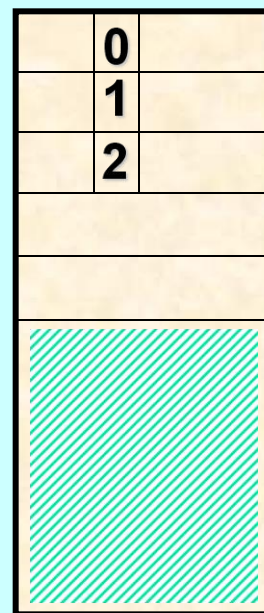
标准文件指针

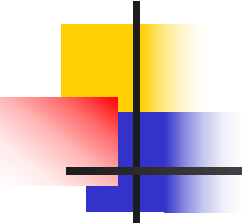
C程序会自动打开3个标准文件。

标准文件	文件指针	默认值
标准输入文件	stdin	键 盘
标准输出文件	stdout	显示器
标准错误文件	stderr	显示器

```
#define stdin    (&_iob[0])  
#define stdout  (&_iob[1])  
#define stderr  (&_iob[2])
```

_iob

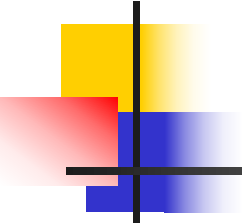




打开方式 mode

r —— 只读 (read)
w —— 只写 (write)
a —— 添加 (append)
+ —— 更新
b —— 二进制文件 (binary)

这些mode符，可以组合；如果其中不含有“b”，则表示以文本文件方式打开。



打开方式 mode

r

以读方式打开文本文件

w

以写方式打开或创建文本文件(清除原内容)

a

以写方式打开或创建文本文件(尾部添加)

rb

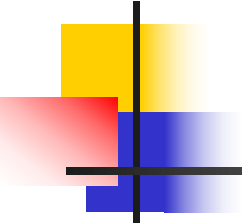
以读方式打开二进制文件

wb

以写方式打开或创建二进制文件(清除原内容)

ab

以写方式打开或创建二进制文件(尾部添加)



打开方式 mode

r+ 以读写方式打开文本文件

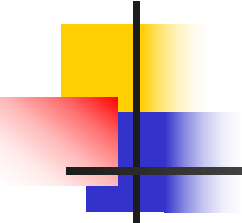
w+ 以读写方式打开或创建文本文件(清除原内容)

a+ 以读写方式打开或创建文本文件(尾部添加)

r+b或rb+ 以读写方式打开二进制文件

w+b或wb+ 以读写方式打开或创建二进制文件(清除原内容)

a+b或ab+ 以读写方式打开或创建二进制文件(尾部添加)



注 意

- ◆ 当指定 “**r +**”、“**w +**” 或 “**a +**” 模式时，读和写都可以。
- ◆ 从读取切换到写入，输入操作必须遇到**EOF**标记，否则，必须调用文件定位函数：**fsetpos**、**fseek** 和 **rewind**。
- ◆ 从写入切换到读取时，必须调用 **fflush** 或文件定位函数。

`fscanf(     );`

`fseek(     );`

`fprintf(     );`

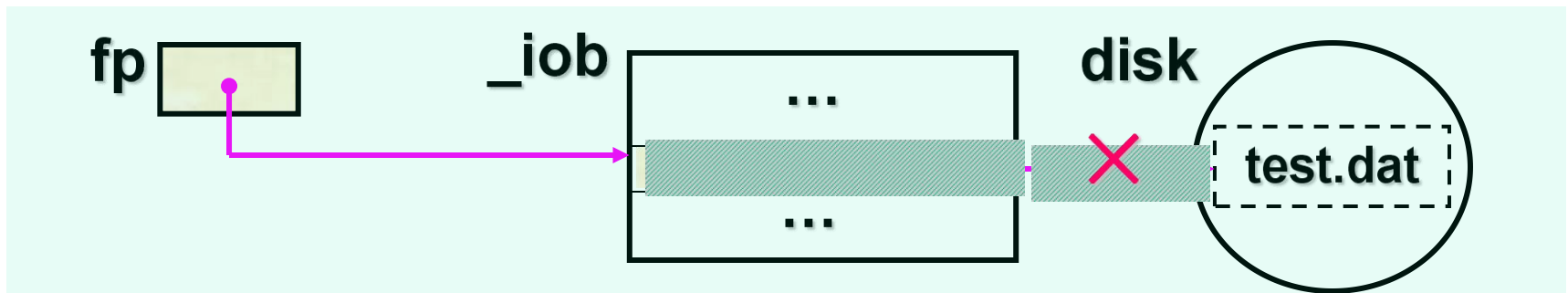
关闭文件函数fclose

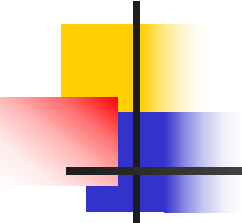
```
int  fclose(FILE *stream);
```

关闭文件指针`stream`“指向”的文件，

正常：返回**0**

出错：**EOF (-1)**。





字符读写函数fgetc和fputc

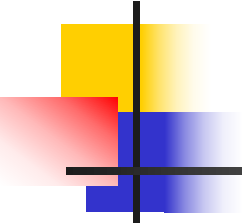
int fgetc(FILE *stream);

从输入流stream当前位置读取一个字符，读写位置后移一个字符，返回读取的字符。到文件尾或读操作出错时返回EOF。

int fputc(int c, FILE *stream);

参数c转换为unsigned char类型然后写到输出流stream的当前位置处。返回值是被写字符；如果写操作出错或遇到文件尾返回EOF。

fgetc(stdin)	等价于	getchar ()
fputc(c, stdout)	等价于	putchar ()



标准输入和输出文件

getchar、gets和scanf函数从stdin文件读数据；

putchar、puts和printf函数向stdout 文件写数据。

freopen函数可以重定向stdin和stdout。如，定向到磁盘文件。

文件的重定向函数freopen

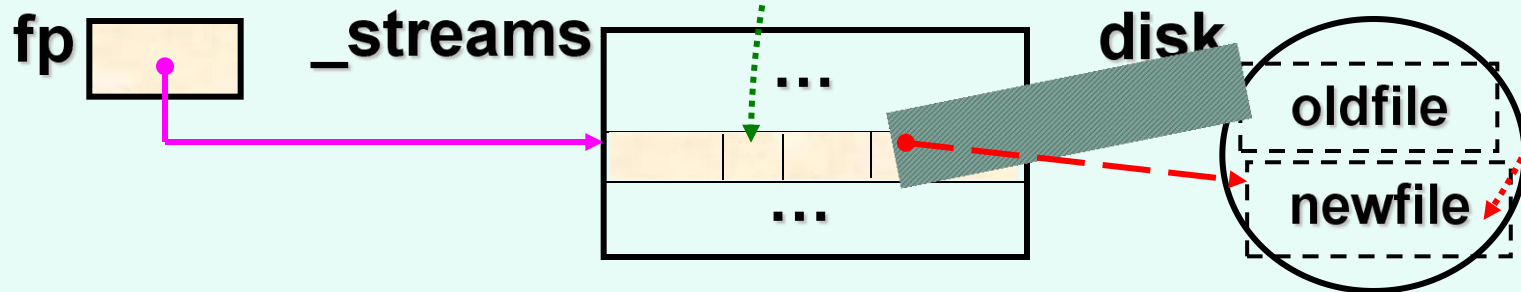
```
FILE *freopen(const char * filename, const char * mode, FILE *fp);
```

(1) fclose (fp) ;

(2) fp=fopen (filename, mode) ;

成功：返回fp

失败：返回NULL。

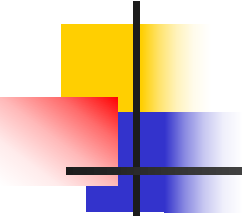




使用freopen改写程序1

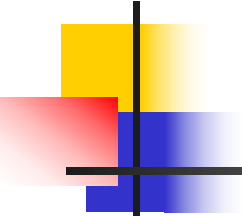
```
#include<stdio.h>
int main( )
{
    char ch;
    if ( (freopen("d:\\a.txt","w", stdout) == NULL){
        printf("can't open the file!");
        return -1;
    }
    while((ch = getchar( ) ) != EOF)
        putchar(ch);

    return 0;
}
```



读取文件中的内容

```
/* 读d:\\a.txt文件中的内容在屏幕上显示出来 */
#include<stdio.h>
int main( )
{
    FILE *fp;
    char ch;
    if ( (fp=fopen( "d:\\a.txt" , "r" )) == NULL) {
        printf("can't open the file!");
        return -1;
    }
    while((ch = fgetc (fp) ) != EOF)
        putchar(ch);
    return 0;
}
```



读取文件中的内容

```
/* 读d:\\a.txt文件中的内容在屏幕上显示出来 */
#include<stdio.h>
int main( )
{
    char ch;
    if ( freopen( "d:\\a.txt" , "r" , stdin) == NULL) {
        printf("can't open the file!");
        return -1;
    }
    while((ch = getchar ( ) ) != EOF)
        putchar(ch);
    return 0;
}
```

字符串读写函数

char * fgets(char *s, int n, FILE *stream);

从stream流中读一行（行长度<n）或至多读n-1个字符到s指向的字符数组中。正常返回s，出错或遇文件尾返回NULL。

char s[10]; // 输入: hust✓

fgets (s, 10, stdin) ; //换行符被读入

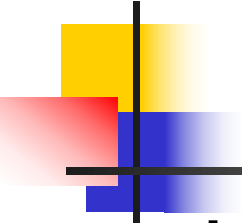
gets(s) ; //换行符不被读入

int fputs(const char *s, FILE *stream);

将指针s 指向的字符串写入stream流中，终结符不写。返回一个非负值；如写操作出错返回EOF。fputs函数与fgets配对使用。

fputs (“hust” , stdout) ; // 输出: hust

puts (“hust”); // 追加输出换行符: hust✓



格式读写函数

int fprintf(FILE *stream, const char *format, ...);

将输出参数列表中的数据按指定的格式写入到**stream**流中。

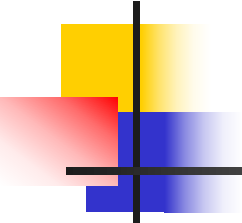
写操作正常返回输出字符个数，写操作出错时返回负值。

int fscanf(FILE *stream, const char *format, ...);

从**stream**流中，按指定的格式读去数据，并赋值给相应的参数变量。函数返回已输入项数，如果读操作出错返回EOF。

fscanf(stdin, " %d" , &x); 等价于 **scanf(" %d" , &x);**

fprintf(stdout, " %d" , x); 等价于 **printf(" %d" , x);**

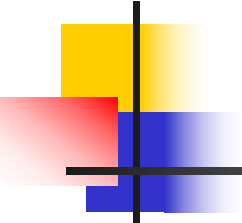


文本文件的复制

演示**copy**命令的功能。

```
c:>copy source_file target_file
```

```
#include<stdio.h>
#include<stdlib.h>
int main(int argc, char* argv[])
{
    int ch;
    FILE *fpin,*fpout;
    if(argc!=3){ /*命令行参数数目不等于3, 说明命令行格式不对*/
        printf("Arguments error!\n");
        exit(-1);
    }
    if((fpin=fopen(argv[1],"r"))==NULL){ /* fpin指向source_file */
        printf("Can't open %s file!\n",argv[1]);
        exit(-1);
    }
    if((fpout=fopen(argv[2],"w"))==NULL){ /* fpout指向target_file */
        printf("Can't open %s file!\n",argv[2]);
        exit(-1);
    }
    while((ch=fgetc(fpin))!=EOF) /* 从source_file中读字符 */
        fputc(ch,fpout); /* 向target_file中写字符, 实现拷贝复制 */
    fclose(fpin); /* 关闭source_file */
    fclose(fpout); /* 关闭target_file */
    return 0;
}
```



文本文件的分解

- 将一个大的文本文件以行为单位分解成为若干个较小的文本文件，文件名和分解的行数都由用户从命令行输入。

将文件abc.txt分解为a.txt, b.txt 和c.txt三个文件，分解的行数为10

parts abc.txt a.txt b.txt c.txt 10

文本文件的分解

parts abc.txt a.txt b.txt c.txt 10

(1) len=命令行中分解的行数

atoi(argv[argc-1])

(2) 以读方式打开源文件

fopen(argv[1], "r")

(3) 依次打开目标文件，从源文件读len行写入目标文件

for (i=2; i<argc-1;i++){

■ 以写方式打开文件 **argv[i]**

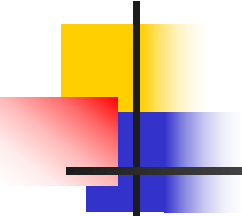
■ 从**argv[1]**读1行写入**argv[i]**直到写了len行或源文件到文件尾。

■ 关闭 文件**argv[i]**

}

(4) 关闭源文件**argv[1]**

```
#include<stdio.h>
#include<stdlib.h>
int main(int argc,char *argv[])
{
    FILE *fin,*fout;
    int len = atoi(argv[argc-1]);/*将行数字字符串转换为整数*/
    int i,j;
    char a[81];
    if((fin = fopen(argv[1],"r")) == NULL){
        printf("can't open the %s file!\n",argv[1]);
        exit(-1);
    }
    for(i = 2; i < argc-1; i++){
        fout = fopen(argv[i],"w");/*打开argv[i]指定的文件进行写操作*/
        j=0;
        while((fgets(a,80,fin) != NULL) && j++<len)/*从fin中读一行到a中*/
            fputs(a,fout);/*将a中字符串写到fout中*/
        fclose(fout);/*写满len行后关闭文件*/
    }
    fclose(fin);
    return 0;
}
```



数据采集和处理程序

数据从键盘输入手工采集，采集商品名称、数量、单价，
计算总金额

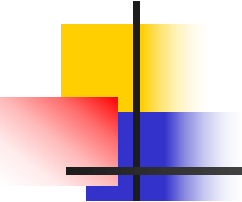
```
#include<stdio.h>
#include <stdlib.h>
void data_write(char *);      /* 数据采集并存盘 */
void data_cal(char *,float ); /* 从文件读入数据并进行计算 */
int main(void)
{
    char a[20] = "d:\\goods.txt";
    data_write(a);
    data_cal(a);
    return 0;
}
```

输入数据并存盘

/*从键盘输入商品名称、数量、单价，存于filename文件中 */

```
void data_write(char * filename)
{
    FILE *out;
    char name[20];
    int number;
    float price;
    if((out = fopen(filename,"w")) == NULL)
        exit(-1);
    printf("input name 、 number and price please!\n");
    while( scanf("%s%d%f",name, &number,&price)!=EOF ) {
        fprintf(out,"%s  %d  %f\n", name, number,price);
    }
    fclose(out);
}
```

必须有空格，便于能够正确进行读



从文件读数据并计算

/*从filename文件中读入商品信息，计算商品的总金额 */

void data_cal(char * filename)

{

FILE * in;

char name[5];

int number;

float price;

if((in = fopen(filename,"r")) == NULL)

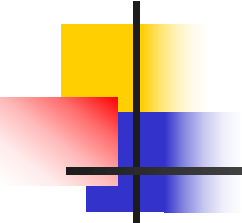
exit(-1);

while(fscanf(in, "%s%d%f" ,name,&number,&price) != EOF)

printf("%s\t%d\t%.2f\n",name,number,price*number);

fclose(in);

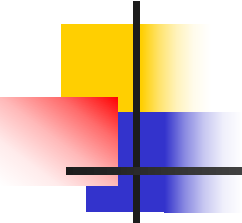
}



文本文件数据的间隔符

- 当将多个数据写入文本文件时，需特别注意的是：在两个数据之间增加必需的**间隔符**。
- 目的在于读文本文件时，能够保证获得数据转换的正确性。

```
fprintf(out,"%s  %d\n",name,number);
```



文件类型

文件按照数据格式分为**文本文件**和**二进制文件**两类。

文本文件 (**ASCII文件**)

由字符的**ASCII**码组成的数据文件

二进制文件

内存表示的数据序列组成的数据文件

文本文件和二进制文件的存储

短整数 $x=128$ 以文本文件和二进制文件在磁盘上分别占多少字节？

内存: x

0000 0000 1000 0000

文本文件

0011 0001 0011 0010 0111 0000

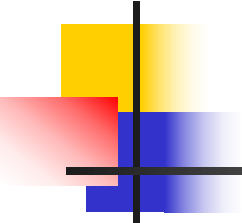
‘1’

‘2’

‘8’

二进制文件

0000 0000 1000 0000



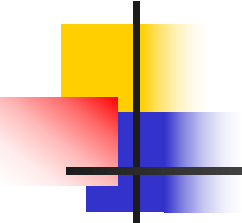
二进制文件的读写

文件直接输入输出又称为文件成组输入输出。标准C为文件的直接输入输出提供了两个函数fread和fwrite，适用于二进制形式文件的读写。

```
size_t fread(void *ptr, size_t size, size_t n, FILE *stream);
```

```
size_t fwrite(const void *ptr, size_t size, size_t n, FILE *stream);
```

```
typedef unsigned int size_t;
```

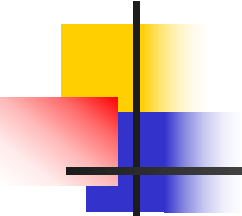


fread函数

`size_t fread(void *ptr, size_t size, size_t n, FILE *stream);`

从stream指向的输入文件中读取至多n个大小为size的记录到指针ptr指向的内存单元中。返回值是实际读取记录数。

文件尾测试函数`feof()`，如果到文件尾，函数返回非0值，否则返回0。`ferror()`测试出错函数，如果出错，`ferror`函数返回非0值，否则返回0。



fwrite函数

```
size_t fwrite(const void *ptr, size_t size, size_t n, FILE *stream);
```

从指针ptr指向的内存缓冲区中取n条，大小为size的记录写到stream指向的输出文件中。返回值是写出的记录数，当返回值小于n时，这种情况只有在写操作出错时出现。

参数的含义同fread，不同的是stream指向的输出文件。

二进制文件写操作

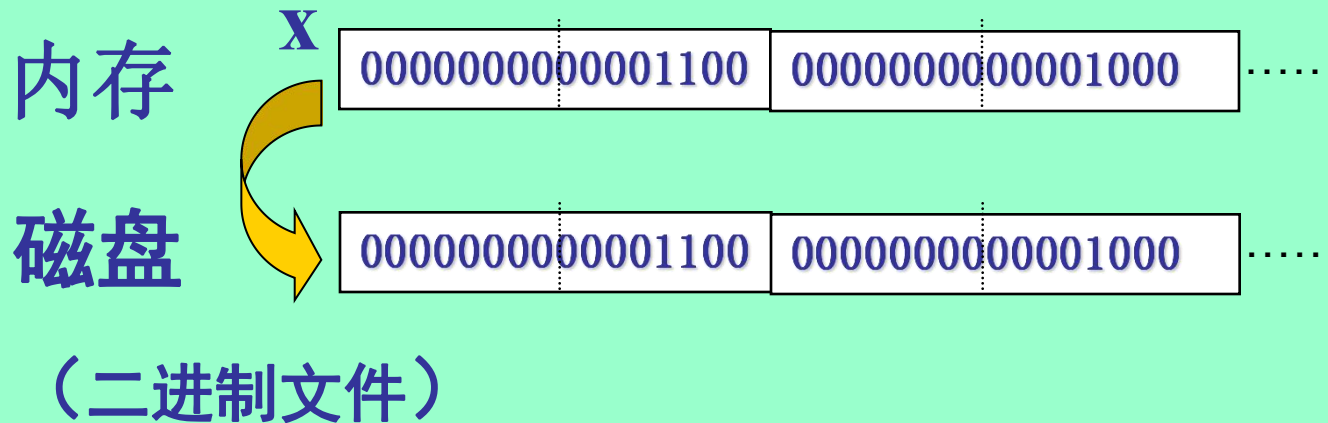
将 `int x[]={12, 8, 34, 421}` 写到磁盘文件

```
fp=fopen("d:\\a.dat","wb");
```

```
(1) fwrite(x, sizeof(int), 4, fp);
```

```
(2) for(i=0;i<4;i++)
```

```
    fwrite(x+i, sizeof(int), 1, fp);
```

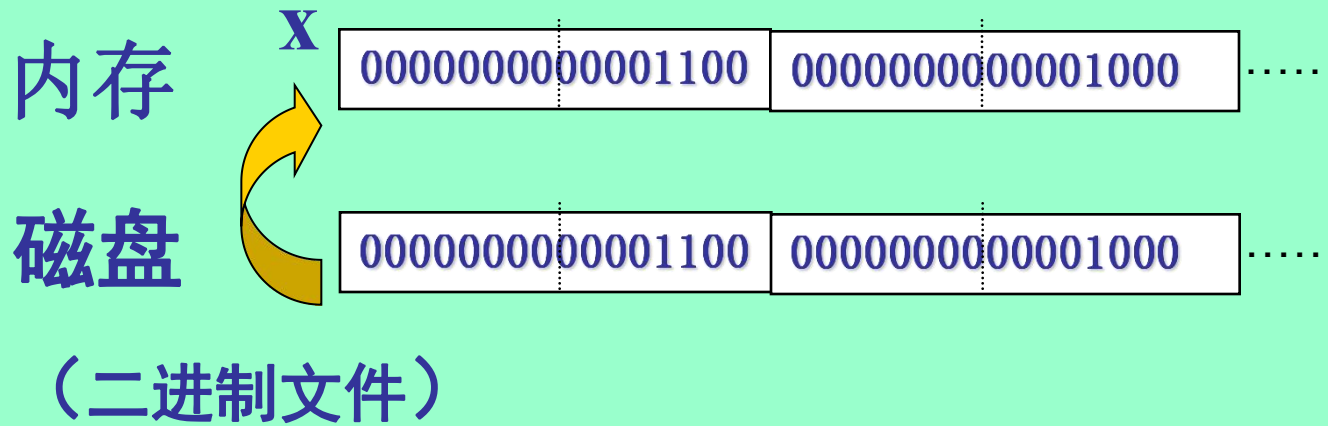


二进制文件数据不需要间隔符

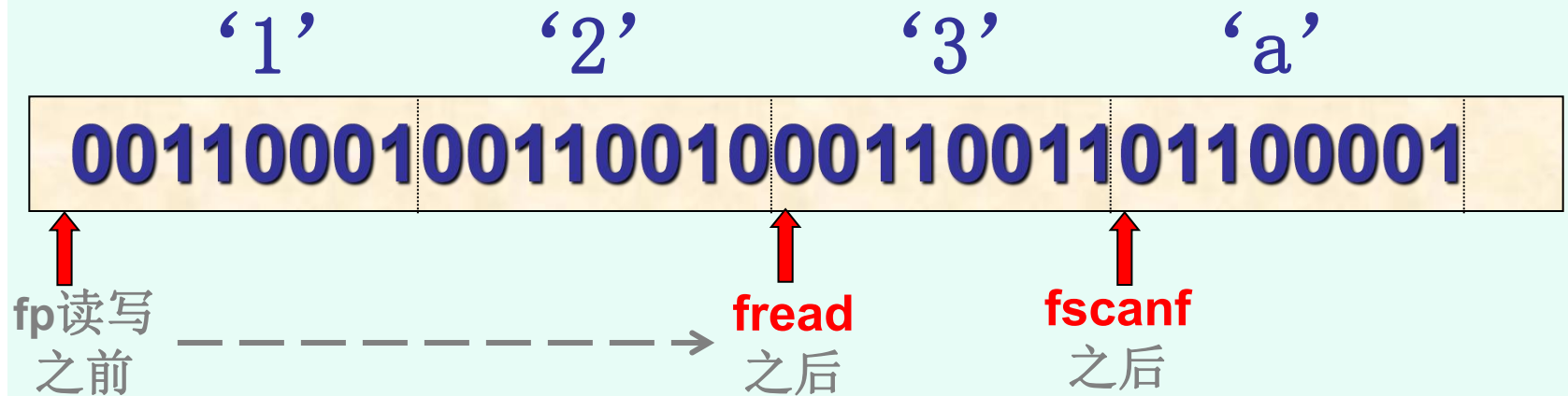
二进制文件读操作

从磁盘文件（二进制）读数据到内存

```
int i=0, x[10]; FILE *fp;  
fp=fopen("d:\\a.dat","rb");  
while( fread(x+i,sizeof(int), 1,fp) ==1) i++;
```



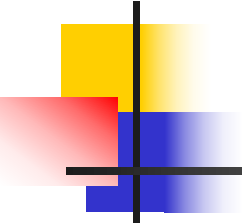
二进制文件数据不需要间隔符



```
short x;
```

```
fread(&x, sizeof(short), 1, fp); //按二进制读入1个short数  
// x=0x3231,低字节在前
```

```
fscanf(fp, "%hd", &x); //按文本格式读入1个short数  
// x=123
```



文件尾测试函数feof()

文件尾测试函数feof()，如果到文件尾，函数返回非0值，否则返回0。

```
#define _IOEOF 0x0010
```

```
#define feof(_stream) ((_stream) -> _flag & _IOEOF)
```

从二进制文件读数据显示到屏幕

```
int x;  
FILE *fp;  
fp=fopen("d:\\a.dat","rb");  
fread(&x,sizeof(int), 1,fp) ;  
while( !feof(fp)) {  
    printf("%d ",x);  
    fread(&x,sizeof(int), 1,fp);  
}
```

只有当文件位置指针到了文件末尾，然后再发生读/写操作时，标志位(fp->_flag)才会被置为含有_IOEOF

所以，要先读，再 feof

从二进制文件读数据显示到屏幕

```
int x;  
FILE *fp;  
fp=fopen("d:\\a.dat","rb");  
while( !feof(fp)) {  
    fread(&x,sizeof(int), 1,fp);  
    printf("%d ",x);  
}
```

最后1个数显示2次

读写商品信息

将例9.5中的物品信息写入磁盘文件

```
#include<stdio.h>
#include<stdlib.h>
/* 声明物品信息结构类型struct goods */
struct goods {
    long code;           /* 货物编码 */
    char name[20];       /* 名称 */
    float price;         /* 价格 */
};

void data_write(const char *filename);
void data_read(const char *filename);

int main(void)
{
    char fname[]="goods_table.dat";
    data_write(fname);
    data_read(fname);
    return 0;
}
```

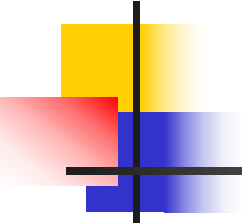
输入商品信息写入磁盘文件

```
void data_write(const char *filename)
{
    struct goods g; /*声明 struct goods 类型结构变量 */
    FILE *out;

    if((out = fopen(filename,"wb")) == NULL)
        exit(-1);
    printf("输入货物编码、名称、价格\n");
    while(scanf("%ld%s%f",&g.code,g.name,&g.price)==3) {
        fwrite(&g,sizeof(struct goods),1,out);
    }
    fclose(out);
}
```

从磁盘读取商品信息显示到屏幕

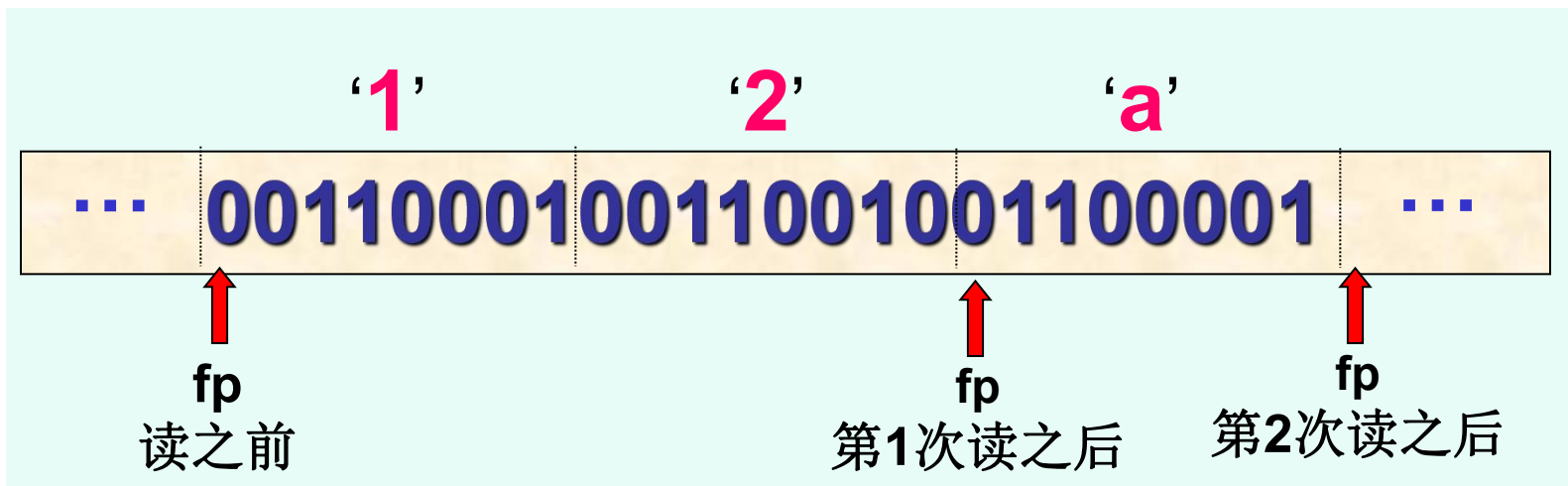
```
void data_read(const char *filename)
{
    struct goods g; /*声明 struct goods 类型结构变量 */
    FILE *in;
    if((in = fopen(filename,"rb")) == NULL)
        exit(-1);
    printf("货物编码\t名称\t价格\n");
    fread(&g,sizeof(struct goods),1,in);
    while(!feof(in)){
        printf("%ld\t%s\t%f\n", g.code,g.name,g.price);
        fread(&g,sizeof(struct goods),1,in);
    }
    fclose(in);
}
```



文件的读写方式

- ◆ 文件的读写方式有两种：**顺序读写**和**随机读写**。
- ◆ 打开文件时，读写指针指向文件头；读写一个“数据”后，读写指针自动指向下一个“数据”。
- ◆ **顺序读写**：从文件头到文件尾顺序读写数据。
- ◆ **随机读写**：对文件的读写可以从文件内指定的位置进行，而不必每次从头顺序开始。

顺序读写



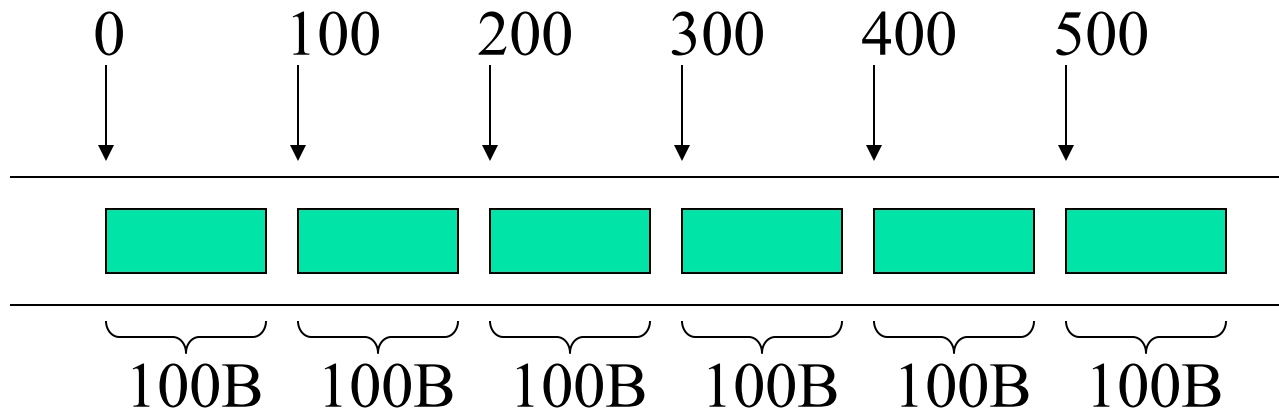
按文本格式，读入1个short数据和1个char数据

```
fscanf(fp,"%hd",&x);    // x:12
```

```
fscanf(fp,"%c",&x);     // x= 'a'
```

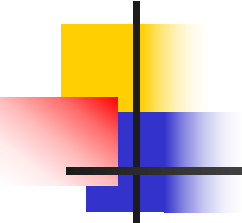
文本文件只能顺序读写，因为其数据不是定长的。

随机读写



具有定长记录的随机存取文件

- 二进制文件数据具有同样的长度，**二进制文件能顺序读写，也能随机读写。**
- 利用文件的定位函数和文件的读写函数，即可实现文件的随机读写。



文件定位函数

int fseek(FILE *stream, long offset, int origin);

long ftell(FILE *stream);

int fgetpos(FILE *stream, fpos_t *pos);

int fsetpos(FILE *stream, const fpos_t *pos);

void rewind(FILE *stream);

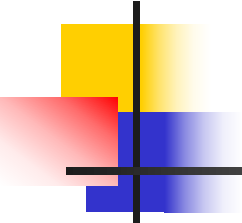
函数fseek

```
int fseek(FILE *stream, long offset, int origin);
```

将stream指向的文件之当前读写指针，定位到origin+offset。如果正常定位，fseek函数返回0，否则返回非0值。

origin和offset分别称为**基准点**和**偏移量**。





函数fseek的基准点

基准点有三种取值：SEEK_SET、SEEK_CUR和SEEK_END

#define SEEK_SET 0 文件起始位置为基准点

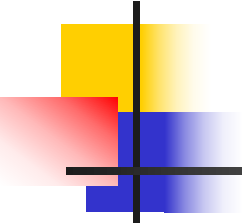
#define SEEK_CUR 1 文件当前位置为基准点

#define SEEK_END 2 文件尾部位置为基准点

- 对二进制文件，基准点的取值与偏移量无关；
- 对文本文件，基准点只能取0，而偏移量可以取0或ftell函数的返回值。

long ftell(FILE *stream);

获取文件读写指针当前位置相对于文件首的偏移字节数，出错时返回-1L。



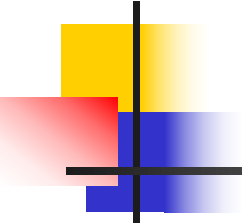
函数fgetpos和fsetpos

```
int fgetpos(FILE *stream, fpos_t *pos);
```

将stream指向文件的读写指针当前值，保存到 pos指针所指的 fpos_t类型的对象中。成功保存，fgetpos返回0，否则返回非0值。

```
int fsetpos(FILE *stream, const fpos_t *pos);
```

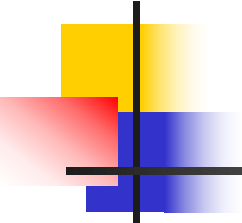
用fgetpos函数保存到pos指针所指对象中的值来设置stream所指向文件的当前位置。设置成功函数返回0，否则返回非0值。



函数rewind

void rewind(FILE *stream);

将文件指针stream指向文件的读写指针重新定位到文件的起始位置，同时清除文件结束标志和出错标志。



其它文件操作函数

int **fflush**(FILE *stream);

int **setvbuf**(FILE *stream, char *buf, int mode, size_t size);

void **setbuf**(FILE *stream, char *buf);

int **remove**(const char * filename);

int **rename**(const char * oldname, const char * newname);

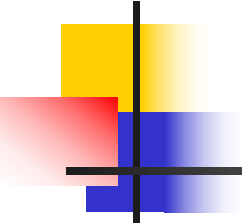
FILE * **tmpfile**(void);

char * **tmpnam**(char *s);

void **clearerr**(FILE *stream);

int **ferror**(FILE *stream);

void **perror**(const char *s);

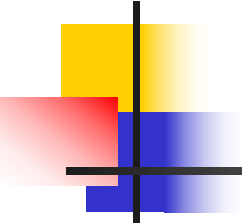


函数fflush

```
int fflush(FILE *stream);
```

将stream指向的输出缓冲区或更新缓冲区（用追加方式打开）中尚未存盘的数据强制性的全部写到介质文件中。如果stream为NULL，执行fflush操作将使所有缓冲区的数据存盘。写操作成功，fflush函数返回0，否则置出错标志并返回EOF。

用于清空文件缓冲区，如果文件是以写的方式打开的，则把缓冲区内容写入文件。

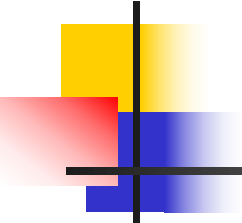


函数setvbuf

```
int setvbuf(FILE *stream, char *buf, int  
mode, size_t size);
```

在打开文件之后，其他任何有关文件的操作被执
行之前调用；setvbuf函数用buf指向的存储区作为
stream指向流的缓冲，缓冲方式由mode参数决定，缓
冲大小由size决定。调用成功，函数返回0，否则返
回非0值。

标准C为mode规定了三种模式：_IOFBF(全缓冲)
、_IOLBF(行缓冲)和_IONBF(无缓冲)。

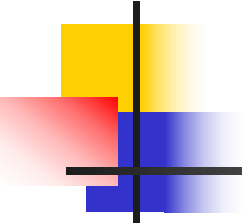


函数setbuf

void setbuf(FILE *stream, char *buf);

功能与 setvbuf函数完全一样。不同的是该函数采用全缓冲(_IOFBF)，缓冲大小用BUFSIZ，函数没有返回值。

缓冲大小值BUFSIZ，在stdio.h定义，一般为512个字节。



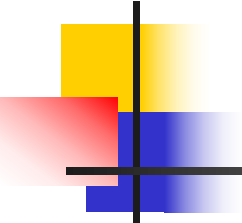
函数remove

```
int remove(const char * filename);
```

删除以filename指向的字符串为文件名的文件。如果操作成功，remove函数返回0，否则返回非0值。

如果对一个已经打开的文件进行remove操作，实际结果由各个C编译器自行决定。

实际上，remove函数就是DOS的del操作之功能。

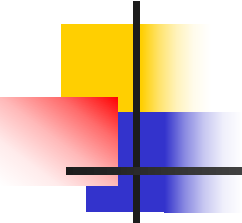


函数rename

```
int rename(const char * oldname, const  
char * newname);
```

以oldname指向的字符串为文件名，更换成以newname指向的字符串为文件名。如果操作成功，函数返回0，否则返回非0值。

实际上，rename函数就是DOS的ren操作之功能。



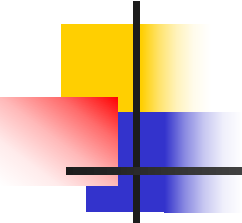
函数tmpfile和tmpnam

FILE * tmpfile(void);

以“wb+”方式创建一个临时二进制文件，该临时文件在文件关闭或程序结束时被自动清除。如果调用成功，tmpfile函数返回一个指向它所创建的临时文件的文件指针，否则返回NULL。

char * tmpnam(char *s);

产生一个与系统中已存在的文件不相同的文件名。当参数非空，函数将产生的文件名存入s指针指向的字符数组，并将s的值作为返回值；如果操作失败，返回一个空指针。



出错检测处理函数

clearerr函数清除stream 所指流的EOF标志和出错标志，无返回值。

ferror函数检测stream所指流的出错，如果出错，ferror函数返回**非0**值，否则返回**0**值。

perror函数将出错信息(全局变量errno的值)写入字符指针s所指的字符数组中，无返回值。